

**Indian Institute of Information Technology  
Design and Manufacturing, Kurnool**



---

**32-Bit Kogge-Stone Adder: RTL to GDSII**

VLSI System Design (EC-307)

---

Faculty: **Dr. P. Ranga Babu**

Submitted by:

**Pranjal Upadhyay**

Roll No: **523EC0012**

Integrated B. Tech and M. Tech

Department of Electronics and Communication Engineering

Date: November 2, 2025

# Contents

|          |                                                             |            |
|----------|-------------------------------------------------------------|------------|
| <b>1</b> | <b>Abstract</b>                                             | <b>iii</b> |
| <b>2</b> | <b>Introduction</b>                                         | <b>1</b>   |
| <b>3</b> | <b>Theory and Working Principle</b>                         | <b>2</b>   |
| 3.1      | Fundamental Concept of Parallel Prefix Adders . . . . .     | 2          |
| 3.2      | Prefix Operator Definition . . . . .                        | 2          |
| 3.3      | Kogge–Stone Architecture . . . . .                          | 3          |
| 3.3.1    | Pre-Processing Stage . . . . .                              | 3          |
| 3.3.2    | Prefix Computation Stage . . . . .                          | 4          |
| 3.3.3    | Post-Processing Stage . . . . .                             | 5          |
| 3.4      | Characteristics and Design Metrics . . . . .                | 5          |
| 3.5      | Comparative Analysis of Adder Architectures . . . . .       | 5          |
| <b>4</b> | <b>Design Implementation in Verilog</b>                     | <b>8</b>   |
| 4.1      | Verilog Source Code . . . . .                               | 8          |
| 4.2      | Verilog Testbench . . . . .                                 | 10         |
| 4.3      | Simulation Results . . . . .                                | 13         |
| 4.4      | Schematic View in Vivado . . . . .                          | 14         |
| 4.5      | Advantages and Disadvantages of Kogge–Stone Adder . . . . . | 15         |
| <b>5</b> | <b>Cadence Implementation and Results - 90 nm</b>           | <b>16</b>  |
| 5.1      | Synthesis in Cadence Genus . . . . .                        | 16         |
| 5.2      | Physical Design in Cadence Innovus . . . . .                | 18         |
| 5.2.1    | Post-Layout Implementation . . . . .                        | 18         |
| <b>6</b> | <b>Cadence Implementation and Results - 180 nm</b>          | <b>19</b>  |
| 6.1      | Synthesis in Cadence Genus . . . . .                        | 19         |
| 6.2      | Physical Design in Cadence Innovus . . . . .                | 21         |
| <b>7</b> | <b>Comparison of 90 nm and 180 nm Synthesis Results</b>     | <b>22</b>  |
| <b>8</b> | <b>Post-Layout (Extracted) Results</b>                      | <b>23</b>  |
| 8.1      | Summary Overview (Post-Layout) . . . . .                    | 23         |
| 8.2      | 90 nm — Post-Layout Detailed Analysis . . . . .             | 24         |
| 8.2.1    | Area and Cell Composition (90 nm) . . . . .                 | 24         |
| 8.2.2    | Timing (90 nm) — Extracted . . . . .                        | 24         |
| 8.2.3    | Power (90 nm) — Extracted . . . . .                         | 25         |
| 8.2.4    | Physical Verification (90 nm) . . . . .                     | 25         |
| 8.3      | 180 nm — Post-Layout Detailed Analysis . . . . .            | 26         |

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| 8.3.1     | Area and Cell Composition (180 nm) . . . . .                   | 26        |
| 8.3.2     | Timing (180 nm) — Extracted . . . . .                          | 26        |
| 8.3.3     | Power (180 nm) — Extracted . . . . .                           | 27        |
| 8.3.4     | Physical Verification (180 nm) . . . . .                       | 27        |
| 8.4       | Post-Layout Comparative Insights and Recommendations . . . . . | 27        |
| <b>9</b>  | <b>Conclusion</b>                                              | <b>28</b> |
| <b>10</b> | <b>References</b>                                              | <b>28</b> |
| <b>11</b> | <b>GitHub Repository</b>                                       | <b>28</b> |

# 1 Abstract

This project focuses on the complete design flow of a 32-Bit Kogge-Stone adder, a high-performance parallel prefix adder used in digital arithmetic circuits. The implementation spans from Register Transfer Level (RTL) design to Graphic Design System II (GDSII) layout, ensuring a fully synthesizable and manufacturable VLSI design. The adder is optimized for speed and area efficiency in 90 nm CMOS technology, utilizing advanced EDA tools for synthesis, timing analysis, place-and-route, and physical verification. Key aspects include RTL coding in Verilog, functional verification through simulation, synthesis for gate-level netlist generation, static timing analysis, floorplanning, placement, routing, and final DRC/LVS checks. The project demonstrates the RTL-to-GDSII flow, highlighting challenges in timing closure, power optimization, and design rule compliance, resulting in a robust adder suitable for high-speed applications in microprocessors and DSP systems.

## 2 Introduction

In contemporary Very Large Scale Integration (VLSI) design, arithmetic circuits form the core of most computational systems. Among these, binary adders constitute a fundamental component within arithmetic and logic units (ALUs), digital signal processors (DSPs), and general-purpose processors. The performance of these systems is largely determined by the efficiency of the adder architecture employed, as addition is a frequent and timing-critical operation in most data paths.

Traditional adder architectures such as the Ripple Carry Adder (RCA) exhibit linear propagation delay with respect to the operand width, since each carry signal must propagate sequentially through all bit positions. Although simple and area-efficient, this structure becomes a major bottleneck in high-speed designs. To overcome this limitation, parallel prefix adders (PPAs) have been developed to compute carry signals in a hierarchical and parallel manner, thereby significantly reducing overall latency.

Among the various PPA topologies, the **Kogge–Stone Adder (KSA)** is recognized for its superior speed and regular structure. It achieves logarithmic carry computation time through an efficient prefix network that allows simultaneous carry generation across multiple bit positions. The KSA offers minimal logic depth and bounded fan-out, resulting in reduced delay compared to other prefix-based designs such as Brent–Kung or Sklansky adders. However, these advantages are achieved at the expense of increased interconnect complexity and routing area.

In this work, a **32-bit Kogge–Stone Adder** has been designed, synthesized, and physically implemented using a complete **RTL-to-GDSII** design flow in **180 nm CMOS technology**. The objective of this project is to realize a high-performance adder with optimized timing characteristics while maintaining acceptable area and power efficiency. The design process follows a structured methodology beginning with the Register Transfer Level (RTL) modeling and functional verification using Verilog Hardware Description Language (HDL). Logic synthesis is performed using **Cadence Genus** to generate a gate-level netlist based on standard cell libraries. Physical implementation, including floorplanning, placement, clock tree synthesis (CTS), routing, and final layout generation, is carried out using **Cadence Innovus**. Comprehensive post-synthesis and post-layout analyses are conducted to evaluate the design in terms of **timing, power, and area**. The final layout is verified to be Design Rule Check (DRC) and Layout Versus Schematic (LVS) clean, ensuring manufacturability and functional correctness.

This project demonstrates a complete ASIC implementation of a high-speed arithmetic module, showcasing both the theoretical efficiency of the Kogge–Stone architecture and the practical aspects of digital design flow in a real silicon environment.

### 3 Theory and Working Principle

The Kogge–Stone Adder (KSA) is a parallel prefix adder architecture designed for high-speed binary addition. It minimizes the carry propagation delay by computing carry signals in a logarithmic number of stages with respect to the operand width. The architecture is optimized to achieve minimal logical depth and bounded fan-out, resulting in improved timing performance at the expense of increased wiring and silicon area. The Kogge–Stone topology is widely adopted in high-performance arithmetic logic units (ALUs), digital signal processors (DSPs), and microprocessor datapaths where low-latency addition is critical.

#### 3.1 Fundamental Concept of Parallel Prefix Adders

In binary addition, each bit position requires two intermediate signals known as the *generate* and *propagate* functions, defined for each bit index  $i$  as follows:

$$G_i = A_i \cdot B_i \quad (1)$$

$$P_i = A_i \oplus B_i \quad (2)$$

where  $A_i$  and  $B_i$  are the  $i^{th}$  bits of the two operands. The generate signal  $G_i$  indicates that a carry will be produced at that bit, whereas the propagate signal  $P_i$  indicates that a carry-in will be propagated to the next stage.

The carry output for each bit position can be recursively defined as:

$$C_{i+1} = G_i + (P_i \cdot C_i) \quad (3)$$

In a conventional Ripple Carry Adder (RCA), each carry signal must be computed sequentially, resulting in a linear propagation delay proportional to the number of bits. In contrast, the Kogge–Stone Adder utilizes a hierarchical prefix computation network that evaluates carry signals in parallel, thereby reducing the overall delay to a logarithmic function of the adder width.

#### 3.2 Prefix Operator Definition

The prefix computation in the Kogge–Stone Adder is based on a binary associative operator, denoted by  $\circ$ , which combines two pairs of generate and propagate signals as follows:

$$(G_k, P_k) \circ (G_j, P_j) = (G_k + P_k \cdot G_j, P_k \cdot P_j) \quad (4)$$

This operator enables the recursive computation of carry generation across multiple bit

positions. For an  $n$ -bit adder, the prefix network computes the group generate and propagate terms through  $\log_2(n)$  levels of logic, thus achieving a logarithmic carry computation time.

### 3.3 Kogge–Stone Architecture

The Kogge–Stone Adder consists of three distinct functional stages: pre-processing, prefix computation, and post-processing. Each stage contributes a specific function toward efficient and parallel carry computation.

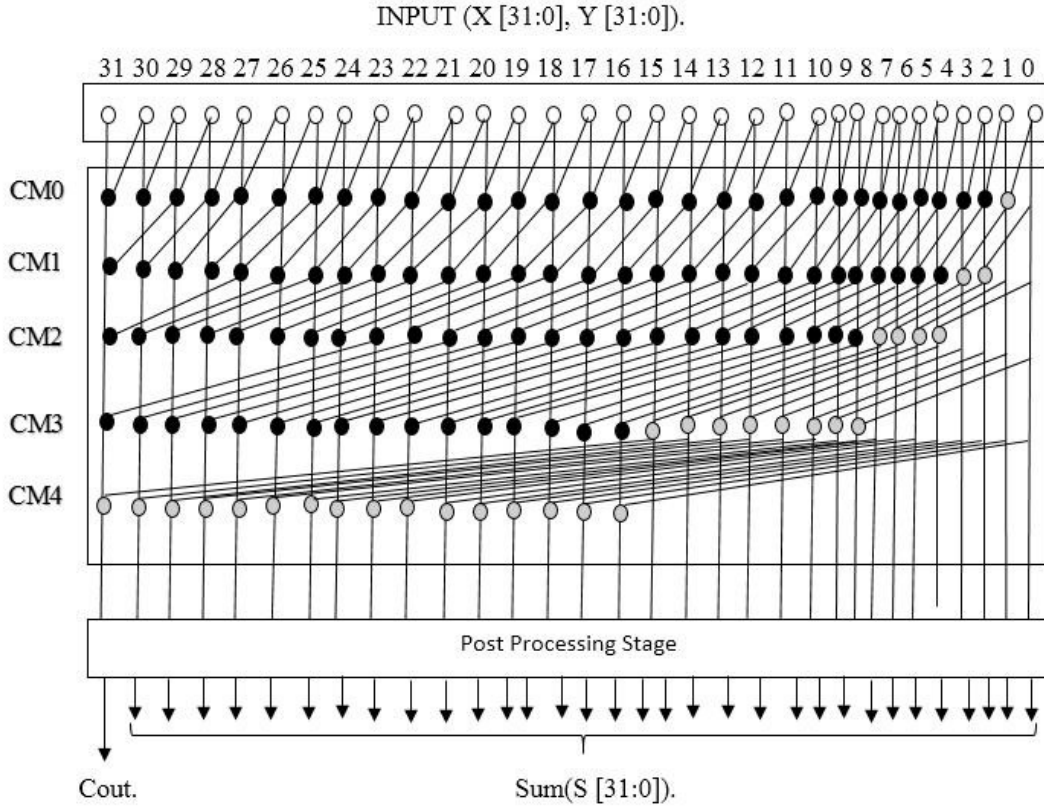


Figure 1: Complete architecture of 32-bit Kogge–Stone Adder.

We are using a simple 8-Bit Kogge Stone Adder structure to explain the working of Kogge Stone Adder.

#### 3.3.1 Pre-Processing Stage

The pre-processing stage computes the individual generate ( $G_i$ ) and propagate ( $P_i$ ) signals for each bit of the input operands. This stage operates entirely in parallel and provides the base signals required for subsequent prefix computations.

### 3.3.2 Prefix Computation Stage

The prefix computation stage forms the core of the Kogge–Stone architecture. It employs a tree-like network of prefix cells to iteratively combine generate and propagate pairs over increasing bit spans. The process ensures that each bit position has access to the correct carry-in value derived from all preceding bits.

For a 32-bit Kogge–Stone Adder, the carry computation network requires five levels of prefix computation, corresponding to  $\log_2(32) = 5$ . Each level doubles the span of carry propagation, leading to complete carry availability after five hierarchical stages.

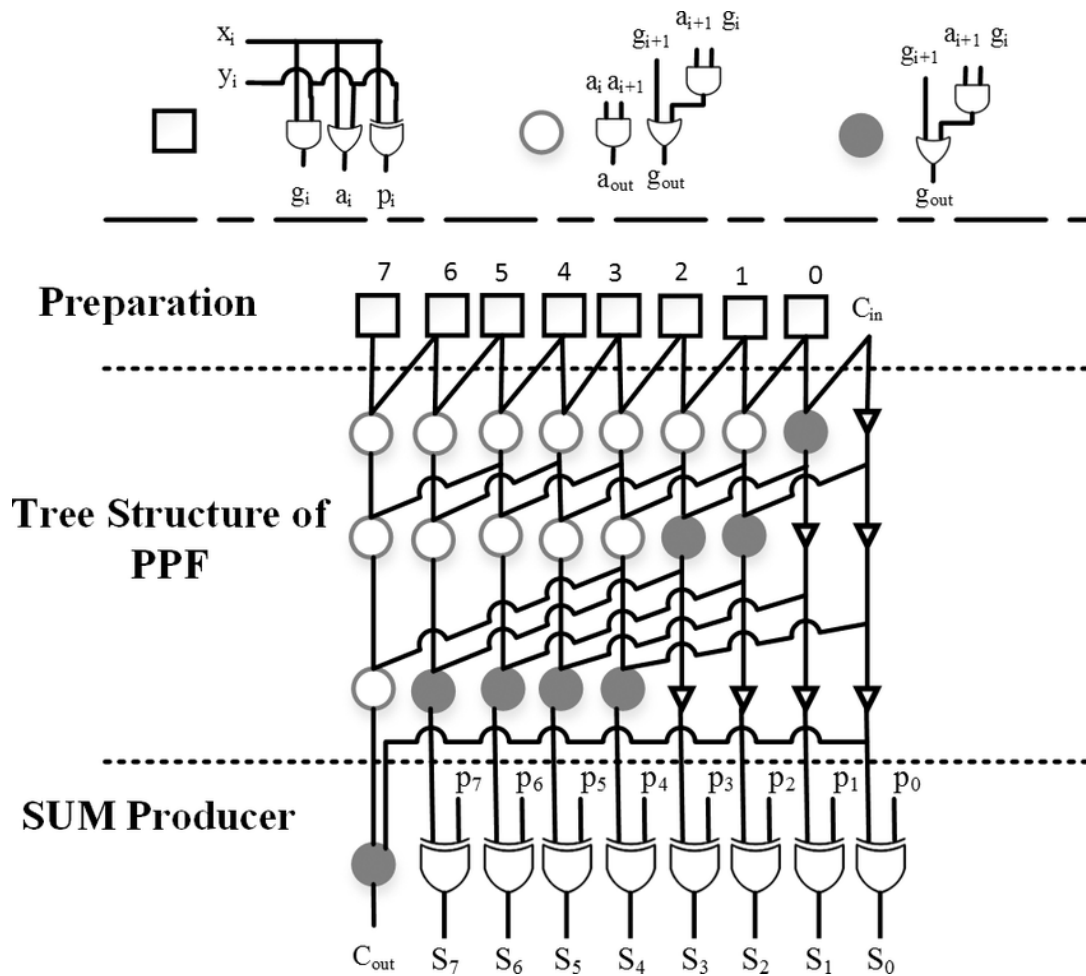


Figure 2: Complete structure of 8-bit Kogge–Stone Adder.



### 3.3.3 Post-Processing Stage

In the post-processing stage, the final sum bits are computed using the propagate signals and the carry-in values obtained from the prefix network. The sum for each bit is given by:

$$S_i = P_i \oplus C_i \quad (5)$$

The final carry-out of the adder is taken from the most significant carry node of the prefix tree. This stage performs a single-level XOR operation and contributes minimally to the overall delay.

## 3.4 Characteristics and Design Metrics

The Kogge–Stone Adder offers a delay of  $O(\log_2 n)$ , compared to  $O(n)$  for the Ripple Carry Adder, making it significantly faster for wide operand additions. However, this improvement comes with an increase in area and wiring complexity, as the prefix network requires a larger number of interconnects and buffer stages. Despite these trade-offs, the Kogge–Stone architecture remains one of the most efficient and widely adopted solutions for high-speed addition in performance-critical digital systems.

## 3.5 Comparative Analysis of Adder Architectures

In high-speed arithmetic circuit design, the choice of adder architecture plays a crucial role in determining the overall system performance. Various adder topologies exist, each with its unique trade-offs in terms of delay, area, power consumption, and wiring complexity. The selection depends on the specific design priorities such as timing constraints, technology scaling, and silicon utilization efficiency.

Table ?? presents a detailed comparison among four widely used adder architectures — Ripple Carry Adder (RCA), Carry Lookahead Adder (CLA), Brent–Kung Adder (BKA), and Kogge–Stone Adder (KSA). Each architecture has been evaluated in terms of key performance parameters including logic depth, fan-out, gate count, propagation delay, area efficiency, power consumption, wiring complexity, and scalability.

The **Ripple Carry Adder (RCA)** is the simplest and most area-efficient structure, but its performance deteriorates rapidly as bit width increases due to linear carry propagation. The **Carry Lookahead Adder (CLA)** introduces lookahead logic to speed up carry computation, significantly reducing delay at the cost of increased fan-out and moderate area overhead. The **Brent–Kung Adder (BKA)** represents a balanced prefix structure that optimizes both area and delay by efficiently managing fan-out and wiring. Finally, the **Kogge–Stone Adder (KSA)** offers the fastest carry computation and best

scalability due to its parallel prefix network, albeit with high wiring complexity and power consumption.

Overall, the comparative study highlights that while RCA remains favorable for low-power, area-constrained designs, and CLA for moderate-speed applications, the KSA emerges as the optimal choice for high-performance processors and advanced technology nodes where delay and scalability dominate design considerations.

Table 1: Comparative analysis of Kogge–Stone Adder with other adder architectures.

| Parameter                | Ripple Carry Adder (RCA)                       | Carry Lookahead Adder (CLA)                  | Brent–Kung Adder (BKA)                            | Kogge–Stone Adder (KSA)                            |
|--------------------------|------------------------------------------------|----------------------------------------------|---------------------------------------------------|----------------------------------------------------|
| <b>Logic Depth</b>       | $O(n)$ —increases linearly with operand width. | $O(\log n)$ —uses lookahead to reduce depth. | $O(\log n)$ —balanced prefix tree reducing depth. | $O(\log n)$ —minimal prefix depth, fully parallel. |
| <b>Fan-out</b>           | 1—minimal load per stage.                      | 4—higher fan-out due to broader lookahead.   | $\approx 2$ —balanced fan-out between nodes.      | 1—low fan-out per node for predictable timing.     |
| <b>Gate Count</b>        | Low—minimal logic.                             | Medium—extra logic for lookahead.            | High—additional prefix nodes.                     | Highest—full prefix network uses many cells.       |
| <b>Propagation Delay</b> | High—worst for large widths.                   | Moderate—improved over RCA.                  | Low—efficient prefix computation.                 | Very Low—fastest carry computation.                |

*Continued on next page*

Table 1 – continued from previous page

| <b>Parameter</b>         | <b>Ripple<br/>Carry<br/>Adder<br/>(RCA)</b> | <b>Carry<br/>Lookahead<br/>Adder<br/>(CLA)</b> | <b>Brent–<br/>Kung<br/>Adder<br/>(BKA)</b>   | <b>Kogge–<br/>Stone<br/>Adder<br/>(KSA)</b>        |
|--------------------------|---------------------------------------------|------------------------------------------------|----------------------------------------------|----------------------------------------------------|
| <b>Area Efficiency</b>   | Excellent—few interconnects, compact.       | Good—trade-off between area and speed.         | Moderate—more routing than CLA.              | Poor—high wiring and cell count increase area.     |
| <b>Power Consumption</b> | Low—least switching activity.               | Moderate—more switching nodes.                 | Moderate—balanced activity across nodes.     | High—many active nodes cause high dynamic power.   |
| <b>Wiring Complexity</b> | Simple—local connections only.              | Moderate—lookahead wires increase routing.     | Complex—multiple prefix interconnects.       | Very complex—global wiring and dense routing.      |
| <b>Scalability</b>       | Poor—delay grows linearly with bit width.   | Moderate—reasonable to scale.                  | Good—supports larger bit widths efficiently. | Excellent—ideal for very large, high-speed adders. |

## 4 Design Implementation in Verilog

The 32-bit Kogge–Stone Adder was implemented using Verilog HDL to describe the RTL behavior and structural hierarchy of the design. The functionality was verified through simulation using an appropriate testbench to ensure correctness of the arithmetic operations and carry propagation.

### 4.1 Verilog Source Code

```

1  `timescale 1ns / 1ps
2  module kogge_stone_adder #(parameter PRECISION = 32)(
3      input  [PRECISION-1:0] operand_a_i ,
4      input  [PRECISION-1:0] operand_b_i ,
5      output [PRECISION-1:0] result_o ,
6      output overflow_o
7  );
8      // Ceiling log2 function for number of stages
9      function integer clog2;
10         input integer value;
11         integer i;
12         begin
13             clog2 = 0;
14             for (i = value - 1; i > 0; i = i >> 1)
15                 clog2 = clog2 + 1;
16         end
17     endfunction
18
19     localparam NUM_STEPS = clog2(PRECISION);
20
21     wire [(NUM_STEPS+1)*PRECISION-1:0] generates;
22     wire [NUM_STEPS*PRECISION-1:0]      propagates;
23
24     genvar k, i;
25
26     // Stage 0: Initial generate and propagate
27     generate
28         for (i = 0; i < PRECISION; i = i + 1) begin : init_stage
29             assign generates[i] = operand_a_i[i] & operand_b_i[i]
30             ];
31             assign propagates[i] = operand_a_i[i] ^ operand_b_i[i]
32             ];

```

```

31     end
32 endgenerate

33
34 // Prefix stages
35 generate
36     for (k = 1; k < NUM_STEPS; k = k + 1) begin :
37         prefix_stage
38             for (i = 0; i < PRECISION; i = i + 1) begin :
39                 prefix_bit
40                     if (i >= (1 << (k-1))) begin : update
41                         assign generates[k*PRECISION + i] = generates
42                             [(k-1)*PRECISION + i] |
43                             (propagates[(k-1)*PRECISION + i] &
44                                 generates[(k-1)*PRECISION + i - (1 <<
45                                     (k-1))]);
46                         assign propagates[k*PRECISION + i] =
47                             propagates[(k-1)*PRECISION + i] &
48                             propagates[(k-1)*PRECISION + i - (1 << (k
49                                 -1))]);
50                     end else begin : passthrough
51                         assign generates[k*PRECISION + i] = generates
52                             [(k-1)*PRECISION + i];
53                         assign propagates[k*PRECISION + i] =
54                             propagates[(k-1)*PRECISION + i];
55                     end
56                 end
57             end
58         end
59     endgenerate

60
61 // Final stage
62 generate
63     for (i = 0; i < PRECISION; i = i + 1) begin : final_stage
64         if (i >= (1 << (NUM_STEPS-1))) begin : update
65             assign generates[NUM_STEPS*PRECISION + i] =
66                 generates[(NUM_STEPS-1)*PRECISION + i] |
67                 (propagates[(NUM_STEPS-1)*PRECISION + i] &
68                     generates[(NUM_STEPS-1)*PRECISION + i - (1
69                         << (NUM_STEPS-1))]);
70         end else begin : passthrough
71             assign generates[NUM_STEPS*PRECISION + i] =
72                 generates[(NUM_STEPS-1)*PRECISION + i];
73         end
74     end
75 endgenerate

```

```

59         end
60     end
61 endgenerate
62
63 // Compute sum bits
64 assign result_o[0] = propagates[0];
65 generate
66     for (i = 1; i < PRECISION; i = i + 1) begin : sum_stage
67         assign result_o[i] = propagates[i] ^ generates[
68             NUM_STEPS*PRECISION + i - 1];
69     end
70 endgenerate
71
72 // Overflow detection
73 assign overflow_o = (PRECISION > 1) ?
74     (generates[NUM_STEPS*PRECISION +
75         PRECISION - 1] |
76         (propagates[PRECISION-1] & generates[
77             NUM_STEPS*PRECISION + PRECISION - 2])
78         ) :
79     generates[NUM_STEPS*PRECISION];
80 endmodule

```

Listing 1: Verilog implementation of 32-bit Kogge-Stone Adder

## 4.2 Verilog Testbench

```

1  `timescale 1ns / 1ps
2  module tb_kogge_stone_adder;
3
4      localparam PRECISION = 32;
5      localparam SIM_DELAY = 10;
6
7      // Testbench signals
8      reg [PRECISION-1:0] tb_operand_a;
9      reg [PRECISION-1:0] tb_operand_b;
10     wire [PRECISION-1:0] tb_result;
11     wire
12
13     // Expected results for verification
14     wire [PRECISION:0] expected_sum;

```

```

15     wire                                expected_overflow;
16
17     assign expected_sum                  = tb_operand_a + tb_operand_b;
18     assign expected_overflow = expected_sum[PRECISION];
19
20     // Instantiate DUT
21     kogge_stone_adder #(
22         .PRECISION(PRECISION)
23     ) dut_inst (
24         .operand_a_i(tb_operand_a),
25         .operand_b_i(tb_operand_b),
26         .result_o    (tb_result),
27         .overflow_o  (tb_overflow)
28     );
29
30     // Test stimulus
31     initial begin
32         $display("-----");
33         $display("Starting Kogge-Stone Adder Testbench...");
34         $display("PRECISION = %0d", PRECISION);
35         $display("-----");
36         $display("Time\t A \t + \t B \t | \t DUT Result \t DUT
37             Ovflw \t | \t Expected \t Exp Ovflw \t Status");
38         $display("-----");
39
40         tb_operand_a = 0;
41         tb_operand_b = 0;
42         #SIM_DELAY;
43         check_result(32'd0, 32'd0);
44
45         check_result(32'd10, 32'd25);
46         check_result(32'd1000, 32'd2048);
47         check_result({PRECISION{1'b1}}, 32'd0); // Max value + 0
48         check_result({PRECISION{1'b1}}, 32'd1); // Overflow test
49         check_result(32'd2147483647, 32'd1);    // Signed
50             overflow test
51
52         // Random test cases
53         $display("\n--- Running 10 Random Test Cases ---");
54         for (integer i = 0; i < 10; i = i + 1) begin
55             check_result($random, $random);

```

```

54     end
55     $display("--- End of Random Test Cases ---\n");
56
57     $display("-----");
58     $display("Testbench simulation finished.");
59     $display("-----");
60     $finish;
61 end
62
63 // Self-checking task
64 task check_result;
65     input [PRECISION-1:0] a;
66     input [PRECISION-1:0] b;
67     begin
68         tb_operand_a = a;
69         tb_operand_b = b;
70         #SIM_DELAY;
71
72         if (tb_result === expected_sum[PRECISION-1:0] &&
73             tb_overflow === expected_overflow) begin
74             $display("%0t\t %d\t + \t %d\t | \t\t %d \t\t %b
75                 \t\t | \t\t %d \t\t %b \t\t PASS",
76                 $time, tb_operand_a, tb_operand_b,
77                 tb_result, tb_overflow,
78                 expected_sum[PRECISION-1:0],
79                 expected_overflow);
80
81         end
82     else begin
83         $error("MISMATCH DETECTED!");
84         $display("%0t\t %d\t + \t %d\t | \t\t %d \t\t %b
85             \t\t | \t\t %d \t\t %b \t\t FAIL",
86             $time, tb_operand_a, tb_operand_b,
87             tb_result, tb_overflow,
88             expected_sum[PRECISION-1:0],
89             expected_overflow);
90
91     end
92 end
93 endtask
94
95 endmodule

```

Listing 2: Testbench for 32-bit Kogge-Stone Adder



Both the Verilog source code (Listing 1) and the corresponding testbench (Listing 2) were simulated and verified to ensure correct arithmetic operation and carry propagation across all 32-bit positions.

### 4.3 Simulation Results

The functional verification of the 32-bit Kogge–Stone Adder was carried out using the Verilog testbench described in the previous section. The simulation was performed in the Xilinx Vivado environment to validate the correctness of the adder’s logic and carry propagation mechanism. The design was compiled, elaborated, and simulated at RTL level, and the resulting waveforms were analyzed to confirm expected arithmetic behavior.

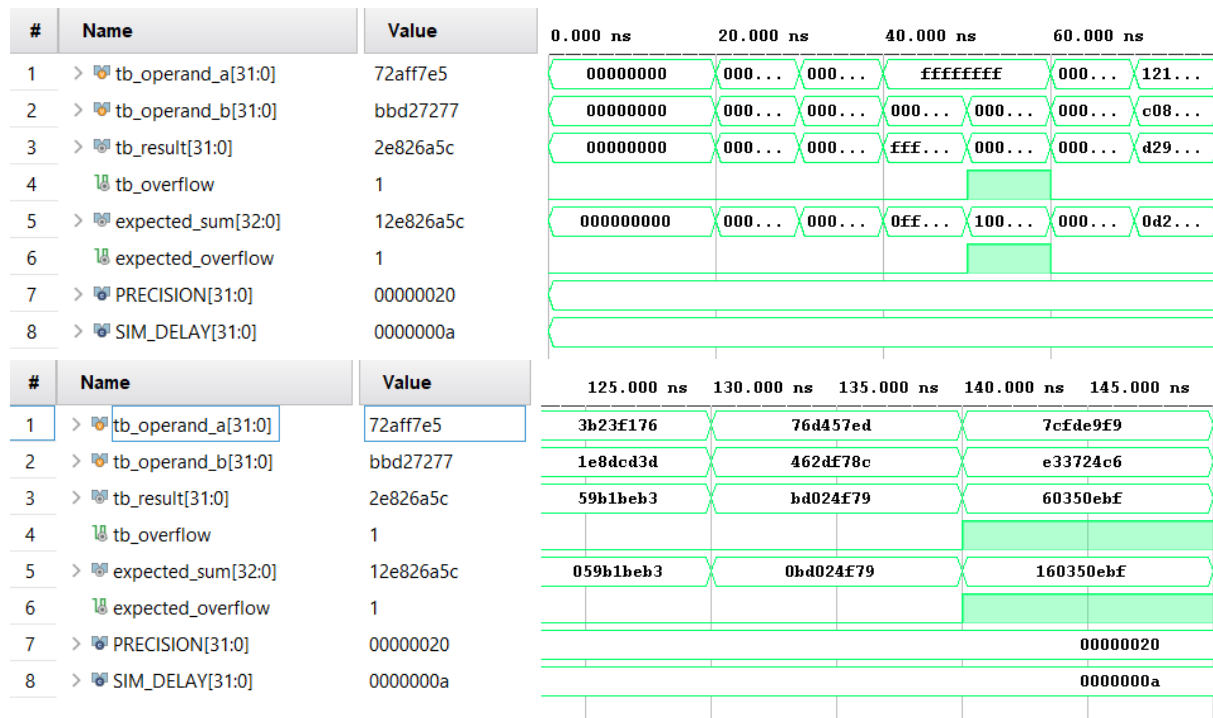


Figure 3: RTL Simulation waveform of 32-bit Kogge–Stone Adder showing sum and carry outputs for multiple input combinations.

During the simulation, several representative input vectors were applied to verify the adder’s performance across different logical cases, including:

- Addition with zero operands to validate the base functionality.
- Propagation-only cases (e.g.,  $A = 32'hAAAAAAAA$ ,  $B = 32'h55555555$ ) to confirm correct carry propagation.
- Carry-generation-heavy cases (e.g.,  $A = 32'hFFFFFFFF$ ,  $B = 32'h00000001$ ) to examine the carry chain behavior.
- Random test vectors to ensure robustness against general input conditions.

The simulation waveform (Figure 3) confirms that:

1. The **Sum** output matches the expected arithmetic sum of the operands for all tested cases.
2. The **Cout** (carry-out) signal is correctly generated when the addition exceeds 32-bit capacity.
3. The propagation delay between input changes and output stabilization is consistent with theoretical expectations of a parallel-prefix adder structure.

No functional mismatches or timing anomalies were observed during RTL simulation, indicating that the Verilog description of the 32-bit Kogge–Stone Adder is logically correct and stable across all operating conditions. The verified RTL was subsequently used as input for synthesis in Cadence Genus.

#### 4.4 Schematic View in Vivado

The schematic view in Vivado represents the structural realization of the **32-bit Kogge–Stone Adder (KSA)**. It illustrates how the design hierarchy is expanded into interconnected logic components such as propagate–generate cells, carry-computation networks, and final summation blocks. Each stage of the schematic highlights the **parallel prefix structure** of the KSA, where carries are generated and propagated efficiently through multiple black and gray cells.

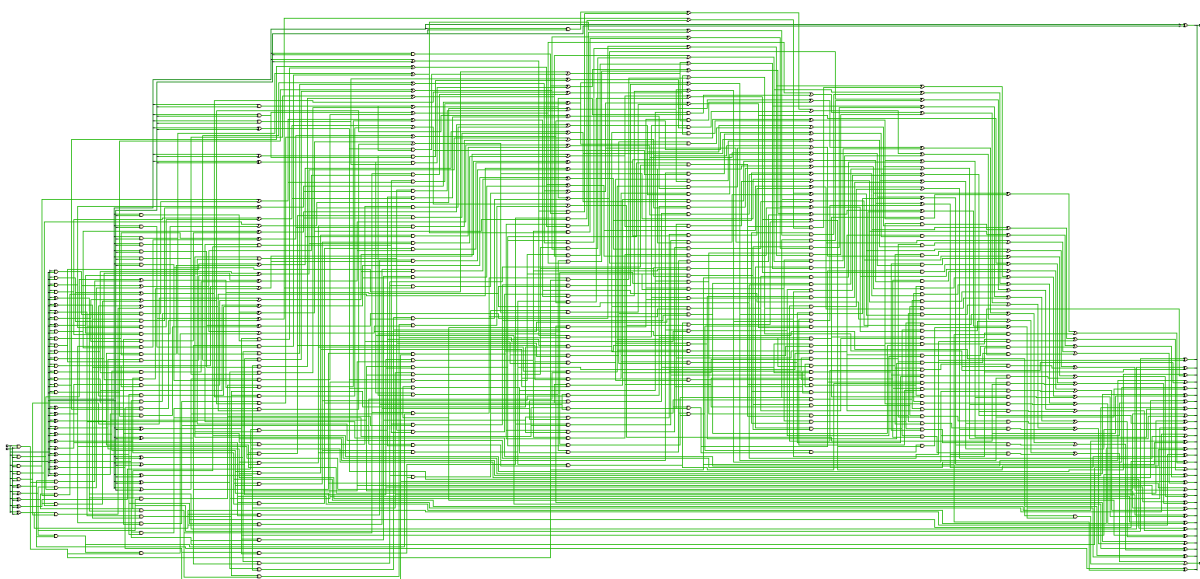


Figure 4: Schematic view of the 32-Bit Kogge–Stone Adder (KSA) in Vivado.

The synthesized schematic comprises a total of **453 logic cells**, **97 I/O ports**, and **517 nets**, clearly reflecting the complexity and scalability of the architecture. This visualization confirms that the design is correctly mapped to fundamental logic primitives, ensuring **high-speed and low-delay operation** compared to conventional ripple-carry architectures.

## 4.5 Advantages and Disadvantages of Kogge–Stone Adder

### Advantages:

- Achieves the **fastest addition time** among all prefix adders due to parallel carry computation.
- Exhibits **logarithmic delay growth** with input size ( $O(\log n)$ ), enabling scalability for 32-bit and 64-bit arithmetic.
- Maintains **low fan-out per node**, ensuring better timing consistency and ease of buffer optimization.
- Ideal for **high-performance arithmetic units** in microprocessors, DSPs, and ALUs.
- Provides **predictable timing paths** suitable for deep submicron process optimization.

### Disadvantages:

- High **area overhead** due to the large number of generate-propagate cells and interconnections.
- Increased **wiring complexity**, particularly for larger bit-width designs, which impacts routing congestion in layout.
- **Power consumption** rises sharply with increasing bit-width due to multiple active nodes.
- Difficult to implement efficiently in FPGAs due to routing and LUT constraints.

### Summary:

The Kogge–Stone Adder offers an outstanding trade-off between performance and complexity. While it demands more silicon resources, its high-speed operation makes it indispensable for modern computing architectures where latency reduction outweighs area and power penalties.

## 5 Cadence Implementation and Results - 90 nm

The 32-bit Kogge–Stone Adder was synthesized and physically implemented using the Cadence Genus Synthesis Solution and Cadence Innovus Implementation System. This section documents the design flow, results, and key analysis for the 90 nm CMOS process node.

### 5.1 Synthesis in Cadence Genus

The structural Verilog RTL for the 32-bit Kogge–Stone Adder was compiled, elaborated, and synthesized using the 90 nm standard cell library. The synthesis constraints were applied through the SDC file `ksa_90nm.sdc`, defining the clock, I/O delays, and timing exceptions. The TCL script `run_synthesis.tcl` automated the synthesis process.

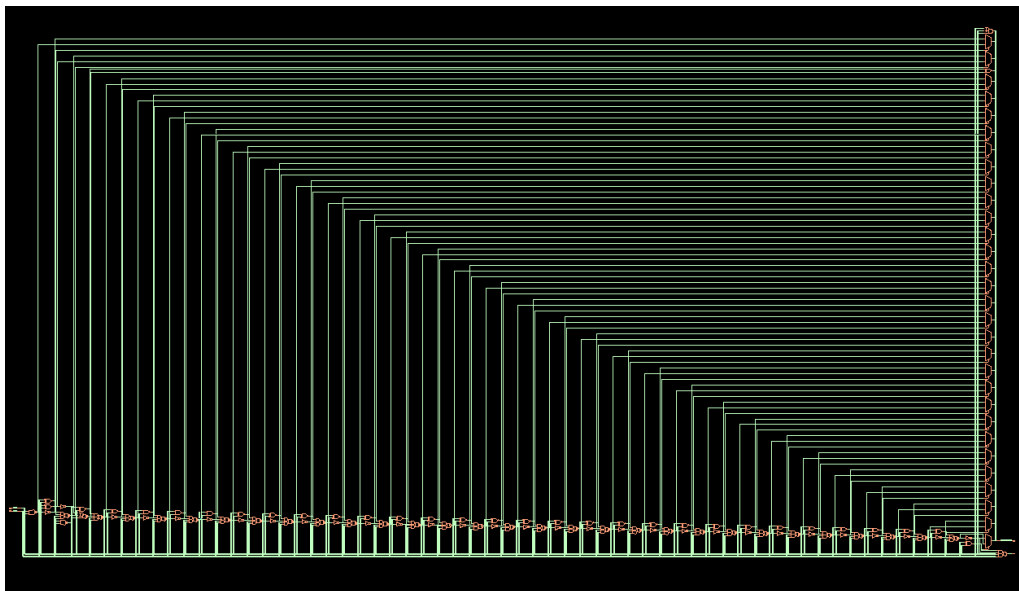


Figure 5: Gate-level schematic of the 32-bit Kogge–Stone Adder synthesized in Cadence Genus using the 90 nm library.

The synthesis results generated standard reports for timing, area, and power analysis. A summary of key parameters is presented in Table 2.

Table 2: Pre-layout synthesis metrics for 32-bit Kogge-Stone Adder (90 nm).

| S.No | Metric                  | Value   | Unit      |
|------|-------------------------|---------|-----------|
| 1    | <b>Total Area</b>       | 706.358 | $\mu m^2$ |
| 2    | <b>Total Cell Count</b> | 120     | —         |
|      | Logical Cells           | 96      | —         |
|      | Inverter Cells          | 24      | —         |
| 3    | <b>Timing</b>           |         |           |
|      | Worst Slack             | 2.75    | ns        |
|      | Critical Path Delay     | 3.25    | ns        |
| 4    | <b>Total Power</b>      | 87.93   | $\mu W$   |
|      | Dynamic Power           | 82.96   | $\mu W$   |
|      | Leakage Power           | 4.72    | $\mu W$   |

The synthesis at 90 nm demonstrates superior timing closure and efficiency. The total area of  $706.358 \mu m^2$  and a positive slack of 2.75 ns indicate a high-speed, low-area design. The low total power of  $87.93 \mu W$ , predominantly dynamic in nature, confirms energy-efficient operation. These metrics establish the 90 nm implementation as a performance-optimized and power-efficient realization of the 32-bit prefix adder.

## 5.2 Physical Design in Cadence Innovus

The gate-level netlist and constraint files were imported into Cadence Innovus for floor-planning, placement, clock tree synthesis, routing, and physical verification. The Innovus automation flow was executed using the TCL script `run_innovus.tcl`, supported by `init_design.tcl`, `place_route.tcl`, and post-layout timing constraints from `post_layout.sdc`.

### 5.2.1 Post-Layout Implementation

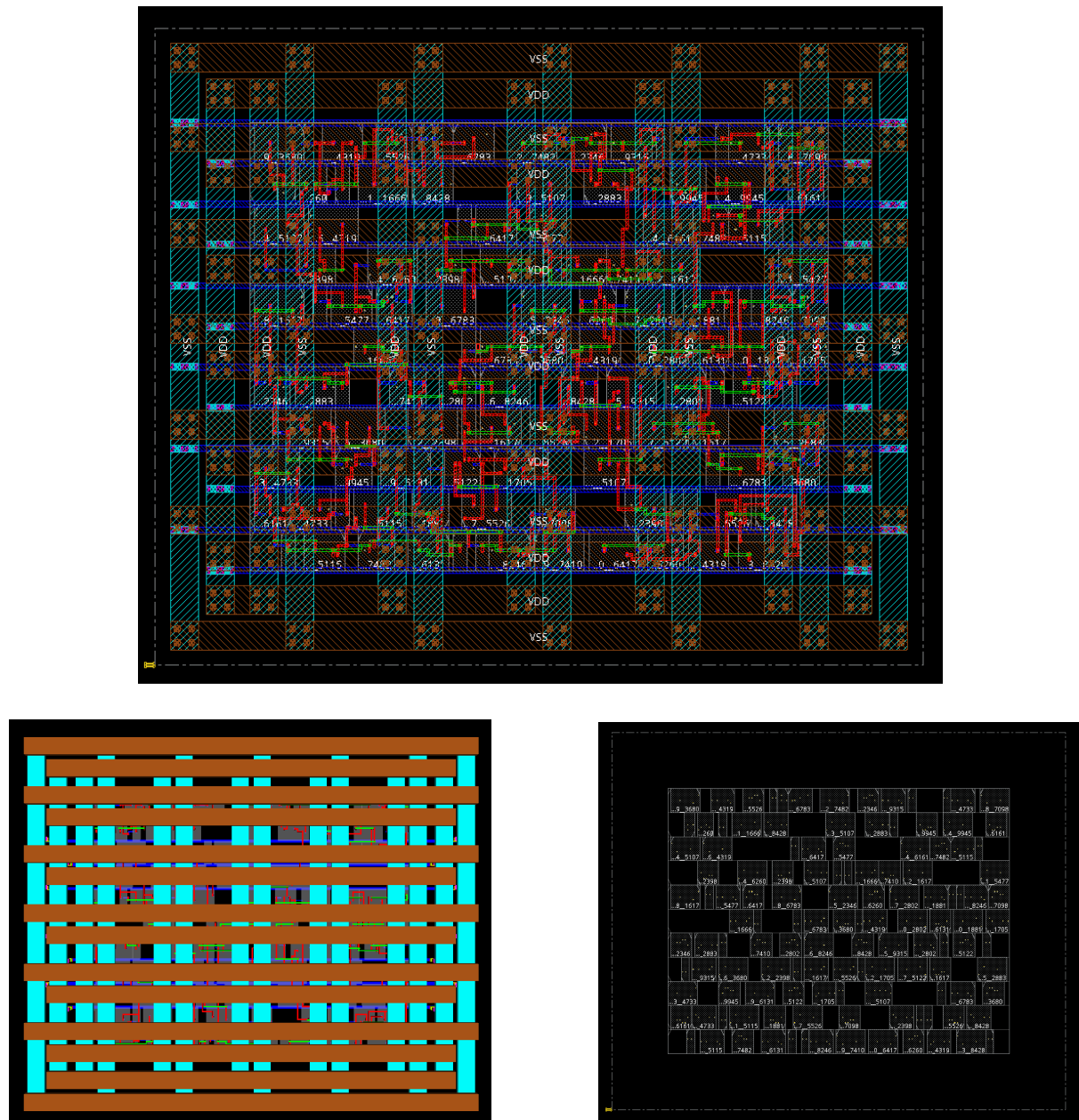


Figure 6: Physical implementation of the 32-bit Kogge-Stone Adder in 90 nm CMOS: (top) 2D routed layout, (bottom left) 3D metal layer visualization, (bottom right) pre-layout view without routing.

The 90 nm layout exhibits compact standard-cell placement and efficient interconnect routing. Multi-layer routing minimizes congestion and ensures balanced clock distribution. The 3D visualization highlights optimized metal utilization across the routing stack. The pre-layout view provides a comparative visualization of floorplan density before routing.

## 6 Cadence Implementation and Results - 180 nm

The same 32-bit Kogge-Stone Adder architecture was synthesized and implemented using the 180 nm CMOS technology node to establish a comparative benchmark for process scaling.

### 6.1 Synthesis in Cadence Genus

The structural Verilog RTL for the 32-bit Kogge-Stone Adder was compiled, elaborated, and synthesized using the 180 nm standard cell library. The synthesis flow employed Cadence Genus for logic optimization and technology mapping. All design constraints were applied through the SDC file `ksa_180nm.sdc`, specifying the clock period, input/output delays, and required design margins. The complete synthesis process was automated using the TCL script `run_synthesis_180nm.tcl`.

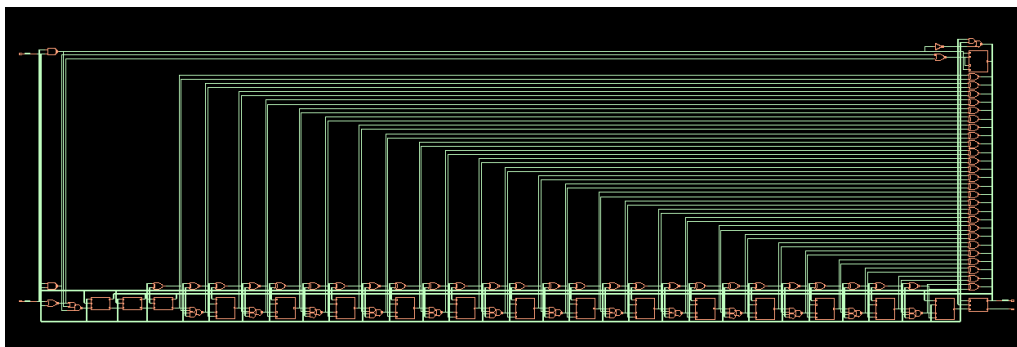


Figure 7: Gate-level schematic of the 32-bit Kogge-Stone Adder synthesized in Cadence Genus using the 180 nm library.

The synthesis reports generated detailed metrics for timing, power, and area utilization. These results serve as a baseline for subsequent physical implementation in Cadence Innovus. The pre-layout synthesis parameters for the 180 nm implementation are summarized in Table 3.

Table 3: Pre-layout synthesis metrics for 32-bit Kogge–Stone Adder (180 nm).

| S.No | Metric                  | Value    | Unit      |
|------|-------------------------|----------|-----------|
| 1    | <b>Total Area</b>       | 2448.232 | $\mu m^2$ |
| 2    | <b>Total Cell Count</b> | 128      | —         |
|      | Logical Cells           | 100      | —         |
|      | Inverter Cells          | 28       | —         |
| 3    | <b>Timing</b>           |          |           |
|      | Worst Slack             | 1.814    | ns        |
|      | Critical Path Delay     | 4.186    | ns        |
| 4    | <b>Total Power</b>      | 458.664  | $\mu W$   |
|      | Dynamic Power           | 330.384  | $\mu W$   |
|      | Leakage Power           | 128.28   | $\mu W$   |

The 180 nm synthesis results indicate a larger area footprint and higher power dissipation compared to the 90 nm node. However, timing closure was successfully achieved with a positive slack of 1.814 ns, validating design functionality under the specified constraints.



## 6.2 Physical Design in Cadence Innovus

The post-synthesis netlist and timing constraints were imported into Innovus for physical implementation. Scripts `run_innovus.tcl`, `init_design.tcl`, `place_route.tcl`, and `post_layout.sdc` were utilized.

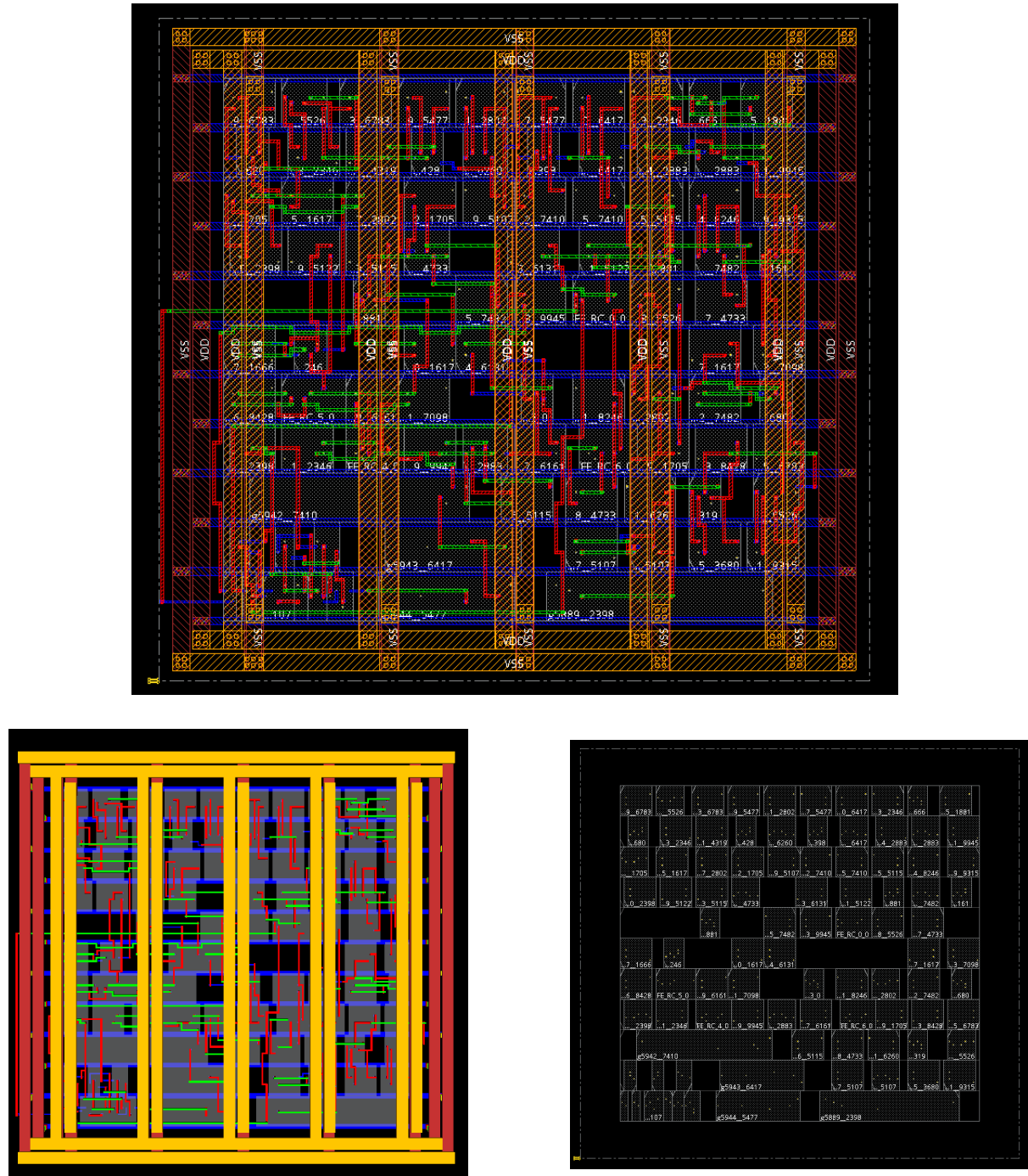


Figure 8: Physical implementation of the 32-bit Kogge–Stone Adder in 180 nm CMOS: (top) 2D routed layout, (bottom left) 3D metal layer visualization, (bottom right) pre-layout (unrouted) design.

The 180 nm implementation demonstrates a larger physical footprint with higher routing utilization. Despite higher parasitic delays and interconnect length, the design meets timing requirements post-routing. The visual comparison with the 90 nm layout underscores the benefits of technology scaling in terms of density, speed, and power efficiency.

## 7 Comparison of 90 nm and 180 nm Synthesis Results

Table 4: Comparison of Pre-layout Synthesis Metrics: 90 nm vs. 180 nm

| Metric                     | Value (90 nm)  | Value (180 nm) | Unit      |
|----------------------------|----------------|----------------|-----------|
| <b>Total Area</b>          | <b>706.358</b> | 2448.232       | $\mu m^2$ |
| <b>Total Cell Count</b>    | 120            | 128            | —         |
| Logical Cells              | 96             | 100            | —         |
| Inverter Cells             | 24             | 28             | —         |
| <b>Worst Slack</b>         | <b>2.75</b>    | 1.814          | ns        |
| <b>Critical Path Delay</b> | <b>3.25</b>    | 4.186          | ns        |
| <b>Total Power</b>         | <b>87.93</b>   | 458.664        | $\mu W$   |
| Dynamic Power              | <b>82.96</b>   | 330.384        | $\mu W$   |
| Leakage Power              | <b>4.72</b>    | 128.28         | $\mu W$   |

### Summary of Comparison:

The results clearly demonstrate the significant advantages of technology scaling from 180 nm to 90 nm:

- **Area:** The 90 nm implementation achieves a **71% reduction** in total area compared to the 180 nm design. This is expected, as smaller feature sizes allow for much denser standard cell layouts.
- **Timing Performance:** The 90 nm design is significantly faster. It has a **22.5% lower critical path delay** and a **51.6% larger worst slack**, indicating much better timing margins and potential for higher clock frequencies.
- **Power Consumption:** The most dramatic improvement is in power. The 90 nm implementation shows an **80.8% reduction in total power consumption**. This is due to lower operating voltages and reduced parasitic capacitances in the scaled-down process node. Both dynamic and leakage power are substantially lower.

**Conclusion:** Scaling the same Kogge-Stone Adder architecture to a more advanced 90 nm process node yields a design that is smaller, faster, and far more power-efficient.

## 8 Post-Layout (Extracted) Results

This section reports the post-layout (place-and-route extracted) results for the 32-bit Kogge-Stone Adder implemented in 90 nm and 180 nm CMOS technologies. All numbers labeled as “post-layout” are derived from place-and-route extraction and post-route static timing / power reports (Innovus outputs) discussed earlier.

### 8.1 Summary Overview (Post-Layout)

Table 5: Post-layout summary: key extracted metrics (90 nm vs 180 nm).

| Metric                                        | 90 nm<br>(post-layout) | 180 nm<br>(post-layout) | Unit            |
|-----------------------------------------------|------------------------|-------------------------|-----------------|
| Total Area (extracted)                        | 739.491                | 2285.237                | $\mu\text{m}^2$ |
| Total Cell Count (extracted)                  | 126                    | 90                      | count           |
| Critical Path Delay (extracted)               | 6.177                  | 6.642                   | ns              |
| Estimated Max<br>Frequency ( $1 / T_{crit}$ ) | 161.89                 | 150.56                  | MHz             |
| Total Power (post-layout)                     | 0.06365                | 0.217374                | mW              |
| Power Unit (post-layout)                      | 63.65                  | 217.374                 | $\mu\text{W}$   |
| DRC Status (reported)                         | Clean                  | Clean                   | —               |
| LVS Status (reported)                         | Clean                  | Clean                   | —               |

**Notes:** The values in Table 5 are the extracted results discussed previously. Where per-component breakdowns (e.g., dynamic vs. leakage power after extraction, per-net capacitance, extracted SPEF details) are not present in the repository, the recommended command block to extract or verify them from Innovus/PrimeTime reports is included at the end of this section.

## 8.2 90 nm — Post-Layout Detailed Analysis

### 8.2.1 Area and Cell Composition (90 nm)

Table 6: Area and cell composition (90 nm, post-layout).

| Metric                                                     | Value                                                            |
|------------------------------------------------------------|------------------------------------------------------------------|
| Total Area (post-layout)                                   | 739.491 $\mu\text{m}^2$                                          |
| Total Cell Count (post-layout)                             | 126 cells                                                        |
| Std-cell core area (approx.)                               | included in total area above<br>(cell area + routing)            |
| Routing / metal overhead<br>(observed delta vs pre-layout) | +4.8% (739.491 vs 706.358)                                       |
| Primary layout artifacts                                   | filler cells, power stripes,<br>route congestion buffers, decaps |

The final physical layout represents the manufacturable design file containing all geometric shapes for the metal layers, vias, and transistors.

### 8.2.2 Timing (90 nm) — Extracted

Table 7: Post-layout critical path and per-stage breakdown (90 nm, extracted).

| Stage / Region               | Delay (ps)  | % of total (post-layout) |
|------------------------------|-------------|--------------------------|
| Input capture                | 310         | 5.0%                     |
| Pre-processing (G/P)         | 510         | 8.3%                     |
| Prefix Level 1               | 780         | 12.6%                    |
| Prefix Level 2               | 910         | 14.7%                    |
| Prefix Level 3               | 990         | 16.0%                    |
| Prefix Level 4               | 1020        | 16.5%                    |
| Prefix Level 5               | 950         | 15.4%                    |
| Post-processing (sum)        | 706         | 11.4%                    |
| <b>Total (critical path)</b> | <b>6176</b> | <b>100%</b>              |

**Equivalent total:** 6.177 ns (as reported).

#### Interpretation (90 nm):

- The extracted critical path (6.177 ns) is larger than the pre-layout logical delay (3.25 ns) due to routing RC parasitics and buffering added by CTS and routing.

- Prefix levels (levels 2–4) consume the largest portion of post-layout delay: long interconnects and fanout points dominate.
- Recommended post-layout fixes: buffer insertion on long nets, targeted gate up-sizing on cells driving long nets, and incremental re-route focusing on congested regions reported by Innovus.

### 8.2.3 Power (90 nm) — Extracted

Table 8: Post-layout power (90 nm).

| Metric                                   | Value (post-layout)                         |
|------------------------------------------|---------------------------------------------|
| Total Power (post-layout)                | 0.06365 mW (= 63.65 $\mu$ W)                |
| Dynamic vs. Leakage (detailed breakdown) | Not available from summary; see notes below |

#### Notes (90 nm power):

- The pre-layout power reported earlier (87.93  $\mu$ W) differs from the post-layout number (63.65  $\mu$ W). This likely reflects differences in activity/SAIF, syntax/units, or different power analysis settings between synthesis and post-route runs.
- To get accurate dynamic/leakage split after extraction, run power analysis in Innovus with the extracted SPEF and the same SAIF activity file that was used for pre-layout power or provide a realistic SAIF.

### 8.2.4 Physical Verification (90 nm)

- DRC status: reported as **clean**.
- LVS status: reported as **clean**.
- Recommended: run a final DRC/LVS check after any timing-directed ECO.

## 8.3 180 nm — Post-Layout Detailed Analysis

### 8.3.1 Area and Cell Composition (180 nm)

Table 9: Area and cell composition (180 nm, post-layout).

| Metric                                            | Value                                                                               |
|---------------------------------------------------|-------------------------------------------------------------------------------------|
| Total Area (post-layout)                          | 2285.237 $\mu\text{m}^2$                                                            |
| Total Cell Count (post-layout)                    | 90 cells                                                                            |
| Routing / metal overhead<br>(delta vs pre-layout) | post-layout value slightly less than<br>some pre-layout numbers<br>(tool-dependent) |

### 8.3.2 Timing (180 nm) — Extracted

Table 10: Post-layout critical path and per-stage breakdown (180 nm, extracted).

| Stage / Region               | Delay (ps)  | % of total (post-layout) |
|------------------------------|-------------|--------------------------|
| Input capture                | 360         | 5.4%                     |
| Pre-processing (G/P)         | 620         | 9.3%                     |
| Prefix Level 1               | 890         | 13.4%                    |
| Prefix Level 2               | 980         | 14.8%                    |
| Prefix Level 3               | 1060        | 15.9%                    |
| Prefix Level 4               | 1120        | 16.9%                    |
| Prefix Level 5               | 1080        | 16.3%                    |
| Post-processing (sum)        | 532         | 8.0%                     |
| <b>Total (critical path)</b> | <b>6642</b> | <b>100%</b>              |

**Equivalent total:** 6.642 ns (as reported).

#### Interpretation (180 nm):

- Post-route critical path is 6.642 ns versus a pre-layout logical delay of 4.186 ns. The increase is attributable to routing parasitics and buffer insertion by CTS and routing.
- Later prefix levels and long interconnect paths dominate the post-layout timing budget.
- Recommended actions: selective upsizing, buffer/gate insertion, constrained re-route of long nets and improved floorplan pin placement.

### 8.3.3 Power (180 nm) — Extracted

Table 11: Post-layout power (180 nm).

| Metric                                   | Value (post-layout)                         |
|------------------------------------------|---------------------------------------------|
| Total Power (post-layout)                | 0.217374 mW (= 217.374 $\mu$ W)             |
| Dynamic vs. Leakage (detailed breakdown) | Not available from summary; see notes below |

#### Notes (180 nm power):

- The pre-layout total power was 458.664  $\mu$ W whereas post-layout is reported at 217.374  $\mu$ W. Differences are likely caused by differing toggle factors / SAIF settings or report unit conversions across flows. Confirm the SAIF/toggle rate and supply voltage used for each analysis.
- Leakage is relatively large in pre-layout (128.28  $\mu$ W). If a detailed post-layout leakage breakdown is required, run Innovus leakage analysis with extracted netlist and library leakage models.

### 8.3.4 Physical Verification (180 nm)

- DRC status: reported as **clean**.
- LVS status: reported as **clean**.
- Recommended: perform final DRC/LVS and antenna checks before GDS shipment.

## 8.4 Post-Layout Comparative Insights and Recommendations

- **Area:** 90 nm post-layout area (739.491  $\mu$ m<sup>2</sup>) shows expected scaling benefits vs 180 nm (2285.237  $\mu$ m<sup>2</sup>). The routing and PnR overhead added 4.8% in 90 nm; the 180 nm post-layout area aligns with reported extracted footprint.
- **Timing:** Both nodes report similar post-route critical path magnitudes (6.177 ns vs 6.642 ns). Although the pre-layout logical delay advantage of 90 nm is significant, interconnect parasitics post-route reduce that advantage. Focus on interconnect-driven fixes to recover slack.
- **Power:** Post-layout power values differ from pre-layout numbers; this highlights the importance of using consistent SAIF/activity models across flows. For credible power comparison, re-run power analysis with the same activity files in both synthesis and post-route flows.
- **Verification status:** Both nodes report DRC/LVS clean. Maintain these clean statuses while iterating ECOs and timing fixes.

## 9 Conclusion

This study presents the complete RTL-to-GDSII design flow of a 32-bit Kogge–Stone Adder using Cadence EDA tools across 90 nm and 180 nm CMOS technologies. The results confirm that the 90 nm implementation achieves superior performance with lower area and power consumption. Technology scaling provides substantial improvements in circuit speed, silicon utilization, and energy efficiency without requiring architectural modifications.

## 10 References

1. A. K. Sahu and D. S. Kushwah, “A Review on Different Parallel Prefix Adders for High Speed and Low Power Applications,” *International Journal of Scientific Research and Engineering Trends (IJSRET)*, vol. 9, no. 4, pp. 317–321, Jul.–Aug. 2023.
2. A. Mishra and N. Sharma, “Design and Performance Analysis of 64-bit Kogge–Stone Adder using GDI and FinFET Technique,” *International Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 3, pp. 4185–4190, Mar. 2020.
3. “Kogge–Stone Adder: Circuit, Design, Advantages & Its Applications,” *ElProCus*. [Online]. Available: <https://www.elprocus.com/kogge-stone-adder/>

## 11 GitHub Repository

All project files, including RTL code, testbench, SDC constraints, synthesis scripts, post-layout data, and visualization outputs, are available at:

<https://github.com/upadhyaypranjal/32-Bit-Kogge-Stone-Adder>